

Изучение рынка заведений общественного питания Москвы

Цели и задачи проекта

Наша цель – провести исследовательский анализ общественного питания в Москве для инвесторов из фонда Shut Up and Take My Money. Эти результаты помогут инвесторам в выборе подходящего места для своего заведения.

Описание данных

Файл `/datasets/rest_info.csv` содержит информацию о заведениях общественного питания:

- `name` — название заведения;
- `address` — адрес заведения;
- `district` — административный район, в котором находится заведение, например Центральный административный округ;
- `category` — категория заведения, например «кафе», «пиццерия» или «кофейня»;
- `hours` — информация о днях и часах работы;
- `rating` — рейтинг заведения по оценкам пользователей в Яндекс Картах (высшая оценка — `5.0`);
- `chain` — число, выраженное 0 или 1, которое показывает, является ли заведение сетевым (для маленьких сетей могут встречаться ошибки):
 - 0 — заведение не является сетевым;
 - 1 — заведение является сетевым;
- `seats` — количество посадочных мест.

Файл `/datasets/rest_price.csv` содержит информацию о среднем чеке в заведениях общественного питания:

- `price` — категория цен в заведении, например «средние», «ниже среднего», «выше среднего» и так далее;
- `avg_bill` — хранит среднюю стоимость заказа в виде диапазона, например:
 - «Средний счёт: 1000–1500 ₽»;
 - «Цена чашки капучино: 130–220 ₽»;
 - «Цена бокала пива: 400–600 ₽» и так далее;
- `middle_avg_bill` — число с оценкой среднего чека, которое указано только для значений из столбца `avg_bill`, начинающихся с подстроки «Средний счёт»:
 - Если в строке указан ценовой диапазон из двух значений, в столбец войдёт медиана этих двух значений.
 - Если в строке указано одно число — цена без диапазона, то в столбец войдёт это число.
 - Если значения нет или оно не начинается с подстроки «Средний счёт», то в столбец ничего не войдёт.
- `middle_coffee_cup` — число с оценкой одной чашки капучино, которое указано только для значений из столбца `avg_bill`, начинающихся с подстроки «Цена одной чашки капучино»:

- Если в строке указан ценовой диапазон из двух значений, в столбец войдёт медиана этих двух значений.
- Если в строке указано одно число – цена без диапазона, то в столбец войдёт это число.
- Если значения нет или оно не начинается с подстроки «Цена одной чашки капучино», то в столбец ничего не войдёт.

Содержимое проекта

1. Загрузка и знакомство с данными
2. Проверка ошибок в данных и предобработка
3. Исследовательский анализ данных
4. Итоговый вывод и рекомендации

1. Загрузка данных и знакомство с ними

Познакомимся с данными и изучим общую информацию о них.

```
In [1]: !pip install phik
```

```
Collecting phik
  Downloading phik-0.12.5-cp39-cp39-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (677 kB)
    |████████████████████| 677 kB 2.3 MB/s eta 0:00:01
Requirement already satisfied: joblib>=0.14.1 in /opt/conda/lib/python3.9/site-packages (from phik) (1.1.0)
Requirement already satisfied: scipy>=1.5.2 in /opt/conda/lib/python3.9/site-packages (from phik) (1.9.1)
Requirement already satisfied: matplotlib>=2.2.3 in /opt/conda/lib/python3.9/site-packages (from phik) (3.3.4)
Requirement already satisfied: numpy>=1.18.0 in /opt/conda/lib/python3.9/site-packages (from phik) (1.21.1)
Requirement already satisfied: pandas>=0.25.1 in /opt/conda/lib/python3.9/site-packages (from phik) (1.2.4)
Requirement already satisfied: pillow>=6.2.0 in /opt/conda/lib/python3.9/site-packages (from matplotlib>=2.2.3->phik) (8.4.0)
Requirement already satisfied: kiwisolver>=1.0.1 in /opt/conda/lib/python3.9/site-packages (from matplotlib>=2.2.3->phik) (1.4.4)
Requirement already satisfied: cyclor>=0.10 in /opt/conda/lib/python3.9/site-packages (from matplotlib>=2.2.3->phik) (0.11.0)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.3 in /opt/conda/lib/python3.9/site-packages (from matplotlib>=2.2.3->phik) (2.4.7)
Requirement already satisfied: python-dateutil>=2.1 in /opt/conda/lib/python3.9/site-packages (from matplotlib>=2.2.3->phik) (2.8.1)
Requirement already satisfied: pytz>=2017.3 in /opt/conda/lib/python3.9/site-packages (from pandas>=0.25.1->phik) (2021.1)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.9/site-packages (from python-dateutil>=2.1->matplotlib>=2.2.3->phik) (1.16.0)
Installing collected packages: phik
Successfully installed phik-0.12.5
```

```
In [2]: # Импортируем библиотеку pandas
import pandas as pd

# Загружаем библиотеки для визуализации данных
import matplotlib.pyplot as plt
import seaborn as sns

# Загружаем библиотеку для расчёта коэффициента корреляции phi_k
from phik import phik_matrix
```

```
In [3]: # Выгружаем данные из датасетов rest_info.csv и rest_price.csv в датафреймы info
info = pd.read_csv('rest_info.csv')
price = pd.read_csv('rest_price.csv')
```

```
In [4]: # Выводим информацию о датафрейме
info.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8406 entries, 0 to 8405
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0    id          8406 non-null   object
1    name        8406 non-null   object
2    category    8406 non-null   object
3    address     8406 non-null   object
4    district    8406 non-null   object
5    hours       7870 non-null   object
6    rating      8406 non-null   float64
7    chain       8406 non-null   int64
8    seats       4795 non-null   float64
dtypes: float64(2), int64(1), object(6)
memory usage: 591.2+ KB
```

```
In [5]: info.head()
```

```
Out[5]:
```

	id	name	category	address	district
0	0c3e3439a8c64ea5bf6ecd6ca6ae19f0	WoWФли	кафе	Москва, улица Дыбенко, 7/1	Северный административный округ
1	045780ada3474c57a2112e505d74b633	Четыре комнаты	ресторан	Москва, улица Дыбенко, 36, корп. 1	Северный административный округ
2	1070b6b59144425896c65889347fcff6	Хазри	кафе	Москва, Клязьминская улица, 15	Северный административный округ
3	03ac7cd772104f65b58b349dc59f03ee	Dormouse Coffee Shop	кофейня	Москва, улица Маршала Федоренко, 12	Северный административный округ
4	a163aada139c4c7f87b0b1c0b466a50f	Иль Марко	пиццерия	Москва, Правобережная улица, 1Б	Северный административный округ

```
In [6]: price.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4058 entries, 0 to 4057
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0    id          4058 non-null   object
1    price       3315 non-null   object
2    avg_bill    3816 non-null   object
3    middle_avg_bill  3149 non-null   float64
```

```
4 middle_coffee_cup 535 non-null float64
dtypes: float64(2), object(3)
memory usage: 158.6+ KB
```

In [7]:

```
price.head()
```

Out [7]:

	id	price	avg_bill	middle_avg_bill	middle_coff
0	045780ada3474c57a2112e505d74b633	выше среднего	Средний счёт:1500–1600 ₽	1550.0	
1	1070b6b59144425896c65889347fcff6	средние	Средний счёт:от 1000 ₽	1000.0	
2	03ac7cd772104f65b58b349dc59f03ee	NaN	Цена чашки капучино:155–185 ₽	NaN	
3	a163aada139c4c7f87b0b1c0b466a50f	средние	Средний счёт:400–600 ₽	500.0	
4	8a343546b24e4a499ad96eb7d0797a8a	средние	NaN	NaN	

Промежуточный вывод

1. Датасет `/datasets/rest_info`

Объём и соответствие описанию:

Датасет `/datasets/rest_info.csv` содержит 9 столбцов и 8406 строк, в которых представлена информация о заведениях общественного питания. Набор столбцов в целом соответствует описанию, однако есть столбец `id` — он в описании не указан.

Пропуски:

В 2 столбцах есть пропуски:

- `hours` : 536 пропусков
- `seats` : 3611 пропусков В остальных столбцах пропусков нет.

Что это значит для анализа:

- `hours` — пропусков немного, но если анализировать режимы работы часть заведений выпадет.
- `seats` — почти половина значений отсутствует и анализ вместимости будет сильно неполным.

Типы данных и корректность:

- `id` , `name` , `category` , `address` , `district` , `hours` — object (логично для текста).
- `rating` — float64 (корректно, рейтинг дробный; диапазон выглядит правдоподобно: от 1.0 до 5.0).
- `chain` — int64 (значения 0/1; можно привести к bool для удобства).

- `seats` — float64 (скорее всего стал float из-за пропусков; по смыслу это целое число, после обработки пропусков можно приводить к целочисленному типу).

2. Datasets /datasets/rest_price

Объём и соответствие описанию:

Датасет `/datasets/rest_price.csv` содержит 5 столбцов и 4058 строк, в которых представлена информация о среднем чеке в заведениях общественного питания.

Набор столбцов в целом соответствует описанию плюс есть столбец `id` — он в описании не указан.

Пропуски:

В 4 столбцах есть пропуски:

- `price` : 743 пропуска
- `avg_bill` : 242 пропуска
- `middle_avg_bill` : 909 пропусков
- `middle_coffee_cup` : 3523 пропуска В столбце `id` пропусков нет.

Что это значит для анализа:

- `middle_coffee_cup` заполнен только у небольшой части заведений (логично: он есть только там, где в `avg_bill` указан капучино)
- `price` и `middle_avg_bill` имеют заметную долю пропусков, а значит анализ среднего чека по районам/категориям будет неполным.

Типы данных и корректность:

- `id`, `price`, `avg_bill` — object (корректно).
- `middle_avg_bill`, `middle_coffee_cup` — float64 (корректно).

Подготовка единого датафрейма

Объединим данные двух датасетов в один, с которым и продолжим работу.

```
In [8]: df = info.merge(price, on='id', how='left')
```

```
In [9]: df.head()
```

```
Out[9]:
```

	id	name	category	address	distri
0	0c3e3439a8c64ea5bf6ecd6ca6ae19f0	WoWfli	кафе	Москва, улица Дыбенко, 7/1	Северный административный окр
1	045780ada3474c57a2112e505d74b633	Четыре комнаты	ресторан	Москва, улица Дыбенко, 36, корп. 1	Северный административный окр
2	1070b6b59144425896c65889347fcff6	Хазри	кафе	Москва, Клязьминская улица, 15	Северный административный окр

	id	name	category	address	distri
3	03ac7cd772104f65b58b349dc59f03ee	Dormouse Coffee Shop	кофейня	Москва, улица Маршала Федоренко, 12	Северный административный окр
4	a163aada139c4c7f87b0b1c0b466a50f	Иль Марко	пиццерия	Москва, Правобережная улица, 15	Северный административный окр

In [10]: `df.shape`

Out[10]: (8406, 13)

In [11]: `# создаем копию датасета до преобразования для возможности проверить сделанный
temp = df.copy()
len(temp)`

Out[11]: 8406

2. Предобработка данных

Подготовим данные к исследовательскому анализу:

In [12]: `# Посмотрим какие типы данных есть в каждом столбце
df.dtypes`

Out[12]:

id	object
name	object
category	object
address	object
district	object
hours	object
rating	float64
chain	int64
seats	float64
price	object
avg_bill	object
middle_avg_bill	float64
middle_coffee_cup	float64
dtype:	object

In [13]: `df['chain'].value_counts()`

Out[13]:

0	5201
1	3205

Name: chain, dtype: int64

In [14]: `# Преобразуем столбец chain из типа данных int64 в тип данных bool
df['chain'] = df['chain'].astype('bool')`

In [15]: `df['category'].value_counts()`

```
Out[15]: кафе                2378
          ресторан          2043
          кофейня           1413
          бар,паб            765
          пиццерия           633
          быстрое питание    603
          столовая           315
          булочная          256
          Name: category, dtype: int64
```

```
In [16]: # Используем метод unique() на category
          df['category'].unique()
```

```
Out[16]: array(['кафе', 'ресторан', 'кофейня', 'пиццерия', 'бар,паб',
                'быстрое питание', 'булочная', 'столовая'], dtype=object)
```

```
In [17]: # Преобразуем столбец category из типа данных object в тип данных category
          df['category'] = df['category'].astype('category')
```

```
In [18]: df['price'].value_counts()
```

```
Out[18]: средние            2117
          выше среднего     564
          высокие           478
          низкие            156
          Name: price, dtype: int64
```

```
In [19]: # Используем метод unique() на price
          df['price'].unique()
```

```
Out[19]: array([nan, 'выше среднего', 'средние', 'высокие', 'низкие'], dtype=object)
```

```
In [20]: # Преобразуем столбец price из типа данных object в тип данных category
          df['price'] = df['price'].astype('category')
```

```
In [21]: # Преобразуем столбец seats из типа данных float64 в тип данных Int64
          df['seats'] = pd.to_numeric(df['seats'], errors='coerce').astype('Int64')
```

```
In [22]: # Понижаем разрядность числовых данных с 64 до 32
          for column in ['rating', 'middle_avg_bill', 'middle_coffee_cup']:
              df[column] = pd.to_numeric(df[column], downcast='float')
```

```
In [23]: df.dtypes
```

```
Out[23]: id                object
          name              object
          category          category
          address           object
          district          object
          hours             object
          rating            float32
          chain             bool
          seats             Int64
          price             category
          avg_bill          object
          middle_avg_bill   float32
```

```
middle_coffee_cup    float32
dtype: object
```

Изучим пропущенные значения в данных:

- посчитаем их количество в каждом столбце датафрейма
- изучим данные с пропущенными значениями и предположим гипотезы их появления
- проведем обработку пропущенных значений.

In [83]:

```
# Выводим количество пропущенных строк в датафрейме
def show_missing_stats(tmp0):

    missing_stats = pd.DataFrame({
        'Кол-во пропусков': tmp0.isnull().sum(),
        'Доля пропусков': tmp0.isnull().mean()
    })
    missing_stats = missing_stats[missing_stats['Кол-во пропусков'] > 0]

    if missing_stats.empty:
        return "Пропусков в данных нет"

    # Форматируем при выводе через Stlyer
    return (missing_stats.style.format({'Доля пропусков': '{:.4f}'}).background_color.applymap(lambda x: 'red' if x > 0.5 else 'blue'))

show_missing_stats(df)
```

Out[83]:

	Кол-во пропусков	Доля пропусков
hours	535	0.0637
seats	3610	0.4297
price	5087	0.6055
avg_bill	4586	0.5458
middle_avg_bill	5253	0.6252
middle_coffee_cup	7867	0.9363

В датасете пропуски встречаются в 6 столбцах (в остальных — отсутствуют):

- **hours** — умеренное число пропусков В датафрейме не указаны часы работы в карточке, либо неполные данные при выгрузке. Из-за этого часть заведений выпадет из анализа режимов работы. Можно оставить NaN, либо заменить индикатором 'unknown' для удобства группировок.
- **seats** — очень много пропусков. Предположительно количество мест часто не указано в Яндекс Бизнес (не заполнили владельцы/пользователи), либо заведение “формата на вынос/точка без зала”, где количество мест не актуально. Анализ вместимости будет неполным; при расчётах по seats выборка сильно уменьшится. Лучше оставить как NaN (не придумывать места), а для сегментаций использовать либо “есть seats / нет seats”, либо работать только по заполненным строкам.
- **price**, **avg_bill** — пропуски заметны, так как не у всех заведений есть ценовая категория/описание среднего чека. Предположительно ценовые данные заполняются не всегда, часть заведений не указала цены, либо формат заведения не предполагает “среднего чека” (например, столовые/кафе на вынос), либо отсутствует карточка/информация в источнике. **Price** часто удобно заменить на

'unknown' (чтобы не терять строки в разрезах по price). `avg_bill` — оставить как есть (это текст), т.к. он не всегда про средний чек.

- `middle_avg_bill` — пропуски закономерны: поле заполняется только если `avg_bill` начинается с «Средний счёт...». Это не ошибка, а логика формирования признака. Оставим NaN.
- `middle_coffee_cup` — очень много пропусков, потому что заполняется только для строк про капучино. Это не ошибка; просто не все заведения — кофейни, и не у всех указана цена капучино. Оставляем NaN.

```
In [27]: # Заполняем пропуски в hours индикатором 'unknown'
df['hours'] = df['hours'].astype('string').fillna('unknown')
```

```
In [28]: # Заполняем пропуски в price индикатором 'unknown'
df['price'] = df['price'].astype('string').fillna('unknown')
```

```
In [29]: df['hours'] = df['hours'].replace('unknown', pd.NA)
df['price'] = df['price'].replace('unknown', pd.NA)
```

```
In [30]: df.head()
```

```
Out[30]:
```

	id	name	category	address	distri
0	0c3e3439a8c64ea5bf6ecd6ca6ae19f0	WoWфли	кафе	Москва, улица Дыбенко, 7/1	Северный административный окр
1	045780ada3474c57a2112e505d74b633	Четыре комнаты	ресторан	Москва, улица Дыбенко, 36, корп. 1	Северный административный окр
2	1070b6b59144425896c65889347fcff6	Хазри	кафе	Москва, Клязьминская улица, 15	Северный административный окр
3	03ac7cd772104f65b58b349dc59f03ee	Dormouse Coffee Shop	кофейня	Москва, улица Маршала Федоренко, 12	Северный административный окр
4	a163aada139c4c7f87b0b1c0b466a50f	Иль Марко	пиццерия	Москва, Правобережная улица, 1Б	Северный административный окр

Проверим данные на явные и неявные дубликаты, например поля с названием и адресом заведения. Для оптимизации проверки нормализуем данные в текстовых столбцах, например с названием заведения.

```
In [31]: # Сделаем проверку явных полных дубликатов до нормализации

rest_info = pd.read_csv('https://code.s3.yandex.net/datasets/rest_info.csv')
rest_price = pd.read_csv('https://code.s3.yandex.net/datasets/rest_price.csv')

df_raw = rest_info.merge(rest_price, on='id', how='left')
```

```
full_duplicates = df_raw.duplicated().sum()
full_duplicates
```

Out[31]: 0

```
In [32]: # Нормализуем текстовые столбцы name и address
df['name'] = df['name'].astype('string').str.strip().str.lower()
df['address'] = df['address'].astype('string').str.strip().str.lower()
```

```
In [33]: # Проверка явных дубликатов по всей строке
df.duplicated().sum()
```

Out[33]: 0

```
In [34]: # Проверяем неявные дубликаты в датафрейме
df.duplicated(subset='id').sum()
```

Out[34]: 0

Сделаем проверку на неявные (неполные) дубликаты после нормализации

```
In [35]: # Проверка неявных дубликатов по сочетанию name и address
name_address_duplicates = df.duplicated(subset=['name', 'address']).sum()
name_address_duplicates
```

Out[35]: 4

```
In [36]: # Удаляем дубликаты
df = df.drop_duplicates(subset=['name', 'address'])
```

```
In [37]: len(df)
```

Out[37]: 8402

- Для дальнейшей работы создаем столбец `is_24_7` с обозначением того, что заведение работает ежедневно и круглосуточно, то есть 24/7:
 - логическое значение `True` — если заведение работает ежедневно и круглосуточно;
 - логическое значение `False` — в противоположном случае.

```
In [38]: df['is_24_7'] = (
    df['hours'].astype('string').str.lower().fillna('')
    .str.contains('ежедневно', na=False)
    &
    df['hours'].astype('string').str.lower().fillna('')
    .str.contains('круглосуточно', regex=True, na=False)
)
df[['hours', 'is_24_7']].head(15)
```

Out[38]:

	hours	is_24_7
0	ежедневно, 10:00–22:00	False

	hours	is_24_7
1	ежедневно, 10:00–22:00	False
2	пн-чт 11:00–02:00; пт,сб 11:00–05:00; вс 11:00...	False
3	ежедневно, 09:00–22:00	False
4	ежедневно, 10:00–22:00	False
5	ежедневно, 10:00–23:00	False
6	пн 15:00–04:00; вт-вс 15:00–05:00	False
7	пн-чт 10:00–22:00; пт,сб 10:00–23:00; вс 10:00...	False
8	ежедневно, 10:00–22:00	False
9	ежедневно, 12:00–00:00	False
10	ежедневно, круглосуточно	True
11	ежедневно, 10:00–21:00	False
12	вт-сб 09:00–18:00	False
13	ежедневно, 08:00–22:00	False
14	ежедневно, 13:00–00:00	False

```
In [84]: df['is_24_7'].value_counts()
```

```
Out[84]: False    7672
         True      730
         Name: is_24_7, dtype: Int64
```

```
In [85]: # Проверим сколько удалено строк датасета
a, b = len(temp), len(df)
print(" Было строк в исходном датасете", a,
      '\n', "Осталось строк в датасете после обработки", b,
      '\n', "Удалено строк в датасете после обработки", a-b,
      '\n', "Процент потерь", round((a-b)/a*100, 2))
```

```
Было строк в исходном датасете 8406
Осталось строк в датасете после обработки 8402
Удалено строк в датасете после обработки 4
Процент потерь 0.05
```

Промежуточный вывод

В ходе предобработки мы объединили данные о заведениях (rest_info.csv) и ценовой информации (rest_price.csv) в один датафрейм по идентификатору id с помощью LEFT JOIN, чтобы сохранить все заведения, даже если по ним нет данных о среднем чеке.

Далее была проверена корректность типов данных и выполнены преобразования:

- столбец chain преобразован из типа int64 в bool;
- столбец category — из object в category;
- столбец seats — из float64 в Int64;
- столбец price — из object в category;

- для числовых столбцов rating, middle_avg_bill и middle_coffee_cup была понижена разрядность числового типа с float64 до более компактного float32.

Текстовые поля были нормализованы: удалены лишние пробелы и приведён единый регистр, что повысило качество группировок и позволило корректно проверить данные на дубликаты. Была выполнена проверка явных и неявных дубликатов; по результатам предобработки из датасета было удалено 4 строки, которые оказались дублирующимися.

Пропуски в hours и price были заменены на явный индикатор unknown, а остальные пропуски (например, в seats и middle_avg_bill) оставлены как отсутствующие значения, чтобы не исказить реальные показатели. Также был добавлен новый признак is_24_7, отражающий круглосуточный режим работы заведения (по словам «ежедневно» и «круглосуточно»).

В результате предобработки размер датасета изменился незначительно:

- в исходных данных было 8406 строк;
- после обработки осталось 8402 строки;
- удалено 4 строки, что составляет 0.05% от исходного объёма данных.

Таким образом, потери данных минимальны, а итоговый датасет подготовлен к дальнейшему исследовательскому анализу.

3. Исследовательский анализ данных

Задача 1

Исследуем какие категории заведений представлены в данных. Исследуем количество объектов общественного питания по каждой категории. Результат сопроводим подходящей визуализацией.

```
In [41]: # Проверяем распределение данных по значениям столбца category
print('Количество объектов общественного питания по каждой категории:')
df['category'].value_counts()
```

Количество объектов общественного питания по каждой категории:

```
Out[41]: кафе                2376
ресторан                2042
кофейня                 1413
бар, паб                 764
пиццерия                 633
быстрое питание         603
столовая                 315
булочная                256
Name: category, dtype: int64
```

Для визуального анализа такого распределения можно использовать столбчатые диаграммы. Визуализируем распределение в столбце category:

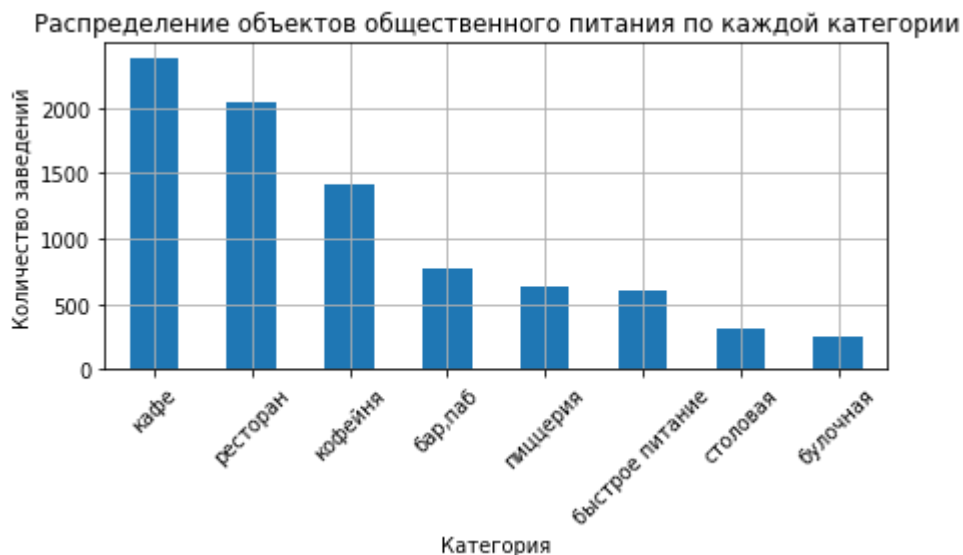
In [42]:

```
# Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(7, 3))

# Строим столбчатую диаграмму с помощью pandas через plot(kind='bar')
df['category'].value_counts().plot(
    kind='bar',
    rot=45,
    legend=False,
    title=f'Распределение объектов общественного питания по каждой
)

# Настраиваем оформление графика
plt.xlabel('Категория')
plt.ylabel('Количество заведений')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



В данных представлены 8 основных категорий заведений общепита, но распределены они неравномерно. Наибольшую долю составляют кафе — 28.3%, рестораны — 24.3% и кофейни — 16.8%. В сумме эти три формата дают около 69.4% всех заведений выборки — то есть рынок в Москве в первую очередь представлен именно “повседневными” форматами, и конкуренция в них будет максимальной.

Остальные категории занимают существенно меньшие доли: бары/пабы — 9.1%, пиццерии — 7.5%, быстрое питание — 7.2%, столовые — 3.7%, булочные — 3.0%. Это означает, что такие форматы либо более нишевые, либо представлены в городе заметно реже. Для инвесторов это даёт два сценария: выход в массовый сегмент (кафе/ресторан/кофейня) потребует сильного позиционирования из-за высокой конкуренции, а выбор нишевой категории (например, булочная или столовая) может означать меньшую плотность конкурентов, но потенциально — более узкую аудиторию и другие требования к локации/формату.

Задача 2

Посмотрим какие административные районы Москвы присутствуют в данных.
Исследуем распределение количества заведений по административным районам
Москвы:

```
In [43]: # Список округов (административных районов)
districts = df['district'].dropna().unique()
print('Округа в данных:', districts)
print('Количество округов:', len(districts))
```

```
Округа в данных: ['Северный административный округ'
'Северо-Восточный административный округ'
'Северо-Западный административный округ'
'Западный административный округ' 'Центральный административный округ'
'Восточный административный округ' 'Юго-Восточный административный округ'
'Южный административный округ' 'Юго-Западный административный округ']
Количество округов: 9
```

```
In [44]: # Абсолютные значения
district_counts = df['district'].value_counts()
print('\nКоличество заведений по округам:')
district_counts
```

Количество заведений по округам:

```
Out[44]: Центральный административный округ      2242
Северный административный округ      898
Южный административный округ      892
Северо-Восточный административный округ      890
Западный административный округ      850
Восточный административный округ      798
Юго-Восточный административный округ      714
Юго-Западный административный округ      709
Северо-Западный административный округ      409
Name: district, dtype: int64
```

```
In [45]: # Относительные значения
district_share = df['district'].value_counts(normalize=True).round(4) * 100
print('\nДоля заведений по округам, %:')
district_share
```

Доля заведений по округам, %:

```
Out[45]: Центральный административный округ      26.68
Северный административный округ      10.69
Южный административный округ      10.62
Северо-Восточный административный округ      10.59
Западный административный округ      10.12
Восточный административный округ      9.50
Юго-Восточный административный округ      8.50
Юго-Западный административный округ      8.44
Северо-Западный административный округ      4.87
Name: district, dtype: float64
```

```
In [46]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(7, 3))

# Строим столбчатую диаграмму с помощью pandas через plot(kind='barh')
df['district'].value_counts().sort_values().plot(
    kind='barh',
    title=f'Количество заведений по административным округам Москв
)

# Настраиваем оформление графика
plt.xlabel('Количество заведений')
```

```
plt.ylabel('Округ')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



Посмотрим распределение заведений каждой категории в Центральном административном округе Москвы.

```
In [47]: # Фильтруем ЦАО
cao_df = df[df['district'] == 'Центральный административный округ']
```

```
In [48]: # Количество заведений по категориям в ЦАО
cao_category_counts = cao_df['category'].value_counts()
print('\nКатегории заведений в ЦАО (количество):')
cao_category_counts
```

Категории заведений в ЦАО (количество):

```
Out[48]: ресторан      670
кафе                  464
кофейня              428
бар, паб             364
пиццерия             113
быстрое питание      87
столовая             66
булочная            50
Name: category, dtype: int64
```

```
In [49]: # Количество заведений по категориям в ЦАО в относительных значениях
cao_category_counts = cao_df['category'].value_counts(normalize = True).round
print('\nКатегории заведений в ЦАО (количество):')
cao_category_counts
```

Категории заведений в ЦАО (количество):

```
Out[49]: ресторан      29.88
кафе                  20.70
кофейня              19.09
бар, паб             16.24
пиццерия             5.04
быстрое питание      3.88
столовая             2.94
булочная            2.23
Name: category, dtype: float64
```

```
In [50]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(7, 3))
```

```
# Строим столбчатую диаграмму с помощью pandas через plot(kind='bar')
cao_category_counts.plot(
    kind='bar',
    title=f'Распределение категорий заведений в ЦАО'
)

# Настраиваем оформление графика
plt.xlabel('Категория')
plt.ylabel('Количество заведений')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



В датасете присутствуют административные округа Москвы (по столбцу district), и распределение заведений по ним неравномерное — есть округа с высокой концентрацией заведений и округа, где объектов заметно меньше. Это ожидаемо для города: плотность заведений обычно выше в районах с высокой деловой/туристической активностью и большим пешеходным трафиком.

Отдельно по ЦАО структура категорий показывает, какие форматы наиболее востребованы в центре: как правило, доля “повседневных” форматов (кафе/кофейни/рестораны) там выше, а более нишевые категории представлены слабее. Такой разрез полезен инвесторам: он помогает понять, насколько рынок в ЦАО конкурентный по конкретным форматам.

Промежуточный вывод

В данных представлены 9 административных округов Москвы, при этом распределение заведений по ним неравномерно. Абсолютный лидер — ЦАО, где сосредоточено 26.7% всех заведений выборки, то есть более четверти рынка. Для сравнения, в большинстве остальных округов доля заведений находится в диапазоне 8–11%, а минимальная концентрация наблюдается в Северо-Западном административном округе — только 4.9%. Это означает, что рынок общепита в Москве явно тяготеет к центру: именно там выше плотность заведений и, вероятно, конкуренция за посетителя.

Структура категорий в ЦАО также отличается от общей картины по Москве. Если в целом по городу лидируют кафе (28.3%), рестораны (24.3%) и кофейни (16.8%), то в центре на первое место выходят уже рестораны (29.9%), а доля кафе снижается до 20.7%. При этом в ЦАО выше доля кофеен (19.1% против 16.8% по городу) и особенно баров/пабов — 16.2% против 9.1% в общей структуре. То есть центр Москвы сильнее смещён в сторону форматов, связанных не только с повседневным спросом, но и с досугом, деловой активностью и более высокой плотностью трафика.

Для инвесторов это означает, что ЦАО — это не просто самый насыщенный округ, а территория с другой структурой спроса и предложения. Если ориентироваться на центр, нужно учитывать более высокую конкуренцию в сегментах ресторанов, кофеен и баров. Если же задача — найти менее перегретую нишу, стоит рассматривать округа вне центра, где плотность заведений ниже, а структура рынка менее смещена в сторону ресторанно-досуговых форматов.

Задача 3

Изучим соотношение сетевых и несетевых заведений в целом по всем данным и в разрезе категорий заведения.

Каких заведений больше — сетевых или несетевых?

```
In [51]: # Соотношение сетевых и несетевых заведений в целом
chain_counts = df['chain'].value_counts(dropna=False)
chain_counts
```

```
Out[51]: False    5199
         True     3203
         Name: chain, dtype: int64
Несетевых заведений: 5201, а сетевых: 3205.
```

```
In [52]: # Соотношение сетевых и несетевых заведений в относительных значениях
chain_counts = df['chain'].value_counts(normalize=True).round(4) * 100
chain_counts
```

```
Out[52]: False    61.88
         True     38.12
         Name: chain, dtype: float64
```

```
In [53]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(7, 3))

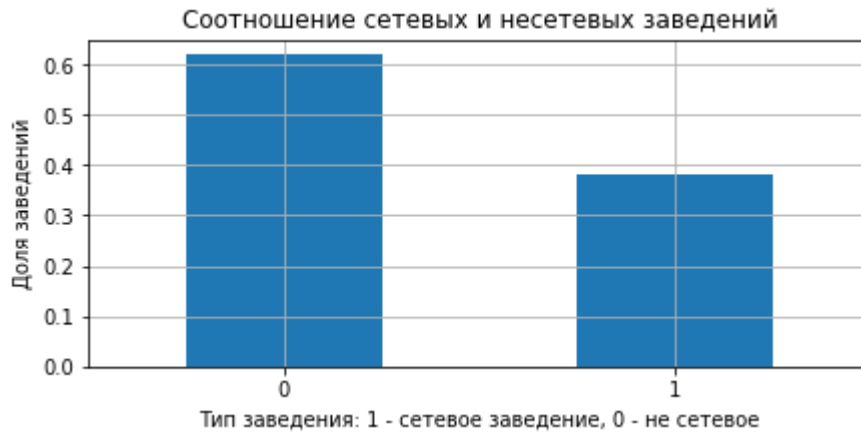
chain_counts = df['chain'].astype(int).value_counts(normalize=True).sort_index()

# Строим столбчатую диаграмму с помощью pandas через plot(kind='bar')
chain_counts.plot(
    kind='bar',
    rot=0,
    title=f'Соотношение сетевых и несетевых заведений'
)

# Настраиваем оформление графика
plt.xlabel('Тип заведения: 1 – сетевое заведение, 0 – не сетевое')
plt.ylabel('Доля заведений')
```

```
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



```
In [54]: # Сетевые/несетевые в разрезе категорий
category_chain_counts = (
    df.pivot_table(index='category', columns='chain', values='id', aggfunc='c'
        .rename(columns={True: 'сетевые', False: 'несетевые'}))
        .sort_values(by=['сетевые', 'несетевые'], ascending=False)
    )

category_chain_counts.head(10)
```

```
Out[54]:
```

category	chain	несетевые	сетевые
кафе		1597	779
ресторан		1313	729
кофейня		693	720
пиццерия		303	330
быстрое питание		371	232
бар, паб		596	168
булочная		99	157
столовая		227	88

```
In [55]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(12, 10))

# Строим столбчатую диаграмму с помощью pandas через plot(kind='bar')
category_chain_counts.plot(
    kind='bar',
    title=f'Сетевые и несетевые заведения по категориям'
)

# Настраиваем оформление графика
plt.xlabel('Категория')
plt.ylabel('Количество заведений')
# Добавляем сетку графика
plt.grid()
```

```
# Выводим график
plt.show()
```

<Figure size 864x720 with 0 Axes>



Какие категории заведений чаще являются сетевыми?

```
In [56]: #Какие категории чаще являются сетевыми
share_chain = (
    df.groupby('category')['chain']
      .mean()
      .sort_values(ascending=False)
)
share_chain.head(10)
```

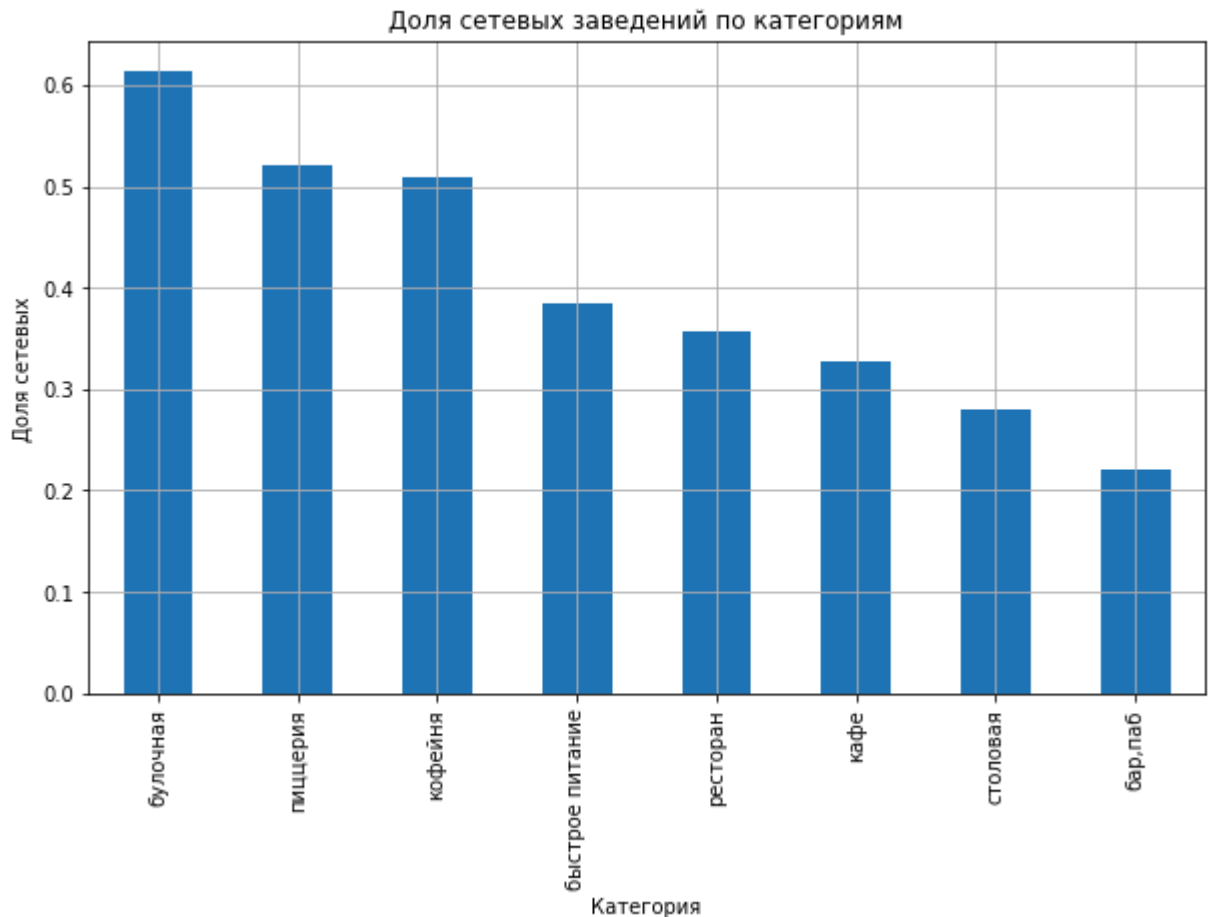
```
Out[56]: category
булочная      0.613281
пиццерия       0.521327
кофейня        0.509554
быстрое питание 0.384743
ресторан       0.357003
кафе           0.327862
столовая       0.279365
бар, паб       0.219895
Name: chain, dtype: float64
```

```
In [57]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(10, 6))

# Строим столбчатую диаграмму с помощью pandas через plot(kind='bar')
share_chain.plot(
    kind='bar',
    title=f'Доля сетевых заведений по категориям'
)

# Настраиваем оформление графика
plt.xlabel('Категория')
plt.ylabel('Доля сетевых')
# Добавляем сетку графика
plt.grid()
```

```
# Выводим график  
plt.show()
```



В целом по выборке несетевых заведений больше, чем сетевых: у большинства заведений `chain=False` (то есть они не входят в сеть), что типично для рынка общепита. В разрезе категорий видно, что чаще всего сетевыми оказываются:

- булочные — 61.33%,
- пиццерии — 52.13%,
- кофейни — 50.96%.

Категории «бар, паб» (22.09%), «столовая» (27.94%), «кафе» (32.76%) и «ресторан» (35.73%) чаще представлены несетевыми заведениями, то есть рынок в этих сегментах более «раздробленный». Это полезно для стратегии: в категориях с высокой долей сетей конкуренция идёт против стандартизированных игроков, а в сегментах с низкой долей сетей проще выделяться уникальной концепцией и сервисом.

Задача 4

Исследуем количество посадочных мест в заведениях. Встречаются ли в данных аномальные значения или выбросы? Если да, то с чем они могут быть связаны? Приведем для каждой категории заведений наиболее типичное для него количество посадочных мест.

```
In [58]: # Изучаем статистические показатели столбца seats
print('Статистические показатели столбца seats:')
df['seats'].describe()
```

Статистические показатели столбца seats:

```
Out[58]: count    4792.000000
mean      108.361436
std       122.841130
min        0.000000
25%       40.000000
50%       75.000000
75%      140.000000
max      1288.000000
Name: seats, dtype: float64
```

По столбцу seats можно сделать вывод, что распределение количества посадочных мест не похоже на нормальное и имеет заметный разброс. Среднее значение (108.4) заметно выше медианы (75), что указывает на правостороннюю асимметрию: в данных есть заведения с очень большим числом мест, которые "подтягивают" среднее вверх. Это подтверждается и высоким стандартным отклонением (122.8) при сравнительно умеренной медиане.

Также разброс значений очень большой: от 0 до 1288 мест. Нулевые значения могут означать либо формат заведений без посадки (например, точки навынос), либо особенности/ошибки заполнения данных. Очень большие значения (сотни и более тысячи мест) могут быть связаны с крупными ресторанами, столовыми или банкетными площадками, а также с возможными выбросами/неточностями в данных. Поэтому для описания "типичного" количества мест в категориях корректнее использовать медиану, а не среднее.

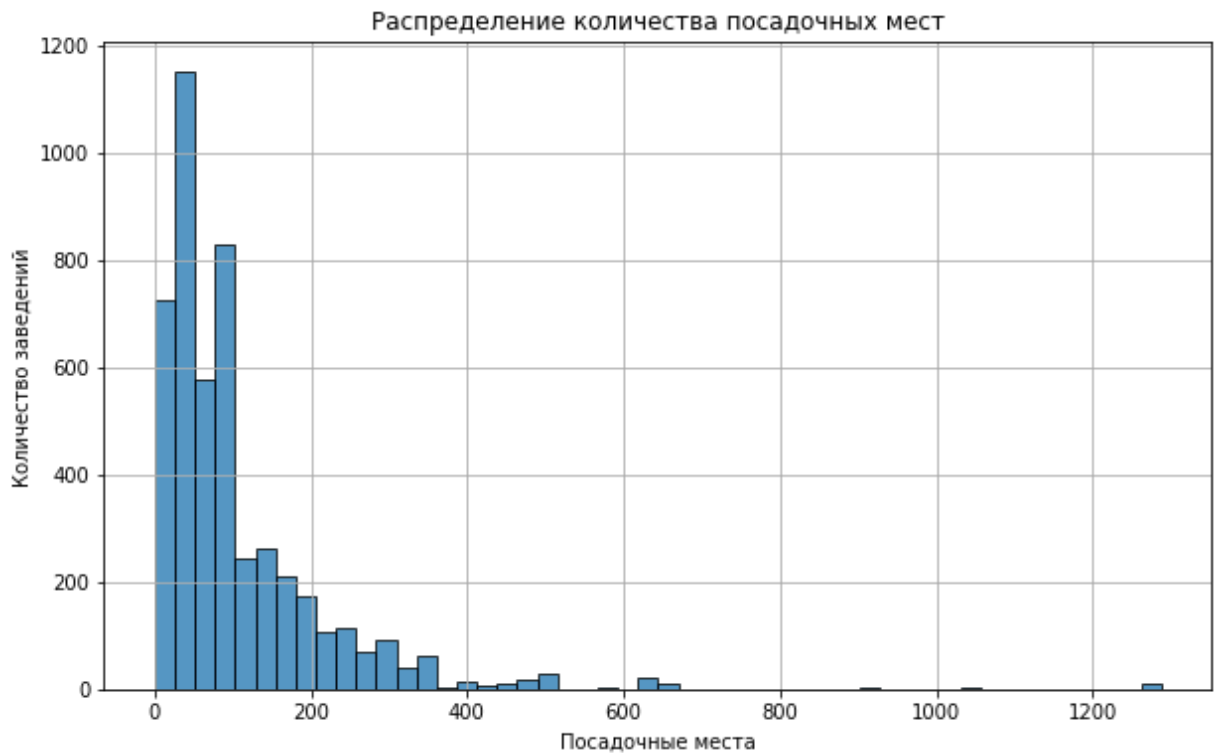
Дополнительно проверим это и построим гистограмму распределения значений и диаграмму размаха для столбца с количеством мест.

```
In [59]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(10, 6))

# Строим гистограмму с помощью pandas через plot(kind='hist')
df['seats'].plot(
    kind='hist',
    bins=50,
    alpha=0.75,
    edgecolor='black',
    rot=0,
)

# Настраиваем оформление графика
plt.title('Распределение количества посадочных мест')
plt.xlabel('Посадочные места')
plt.ylabel('Количество заведений')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



```
In [60]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(7, 2))

# Строим диаграмму размаха значений в столбце balance
df.boxplot(column='seats', vert=False)

# Добавляем заголовок и метки оси
plt.title('Распределение количества посадочных мест')
plt.xlabel('Посадочные места')

# Выводим график
plt.show()
```

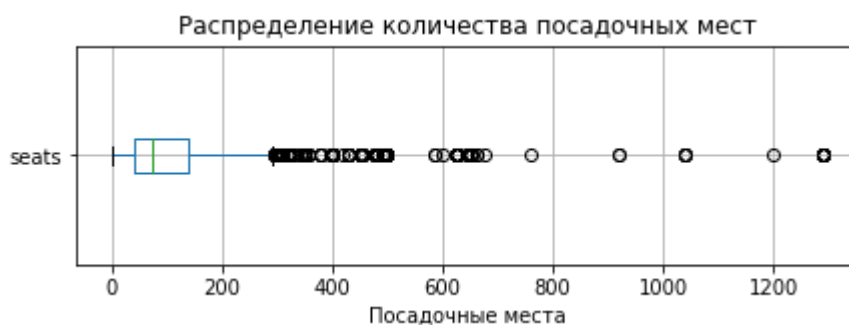


График показывает, что количество посадочных мест распределено неравномерно: большинство заведений имеет относительно небольшое число мест, но в данных присутствует много выбросов с очень большой вместимостью. Поэтому при анализе вместимости и сравнении категорий заведений целесообразно ориентироваться на медиану и квартильный размах, а экстремальные значения анализировать отдельно.

По этому boxplot можно сделать такие выводы: Распределение seats сильно скошено вправо (правосторонняя асимметрия): основная масса заведений имеет сравнительно небольшое/среднее число посадочных мест, а справа виден длинный "хвост" из больших значений.

Есть много выбросов — на графике видно большое количество точек правее верхней границы “усов” (примерно после 250–300 мест). Значит, встречаются заведения с необычно большим количеством мест.

Типичное значение (медиана) находится заметно левее крупных значений, то есть для описания “обычного” заведения лучше использовать медиану, а не среднее.

Видны экстремально большие значения (около 600, 800, 900, 1000+ и до 1288 мест) — это могут быть: крупные рестораны/банкетные площадки/столовые либо ошибки/особенности заполнения данных.

Также на графике присутствуют очень маленькие и нулевые значения, что может указывать на заведения без посадки (формат навынос) или пропуски/неточности, записанные как 0.

```
In [61]: # Типичное количество посадочных мест по категориям (медиана)
typical_seats_by_category = (
    df.groupby('category')['seats']
      .median()
      .sort_values(ascending=False)
)

typical_seats_by_category
```

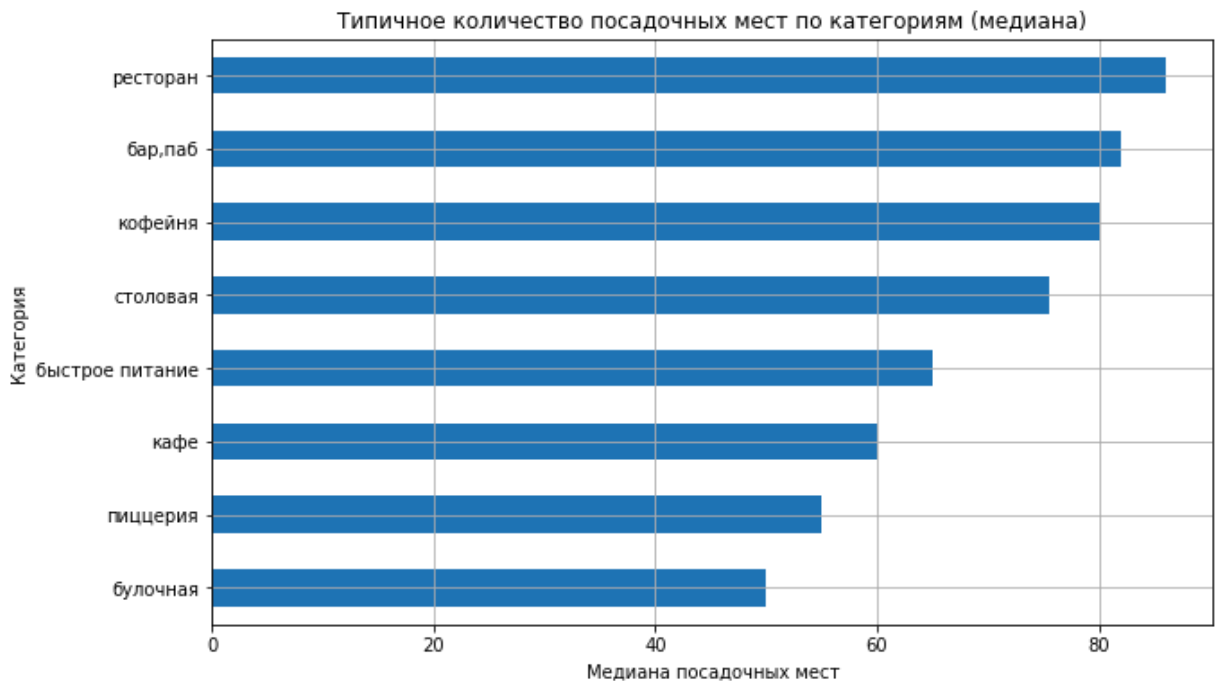
```
Out[61]: category
ресторан      86.0
бар, паб      82.0
кофейня       80.0
столовая      75.5
быстрое питание 65.0
кафе          60.0
пиццерия      55.0
булочная     50.0
Name: seats, dtype: Float64
```

```
In [62]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(10, 6))

# Строим столбчатую диаграмму с помощью pandas через plot(kind='barh')
typical_seats_by_category.sort_values().plot(
    kind='barh',
    title=f'Типичное количество посадочных мест по категориям (мед
)

# Настраиваем оформление графика
plt.xlabel('Медиана посадочных мест')
plt.ylabel('Категория')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



Поскольку распределение `seats` имеет много выбросов, типичное число посадочных мест для каждой категории определялось по медиане, а не по среднему.

По медианному числу посадочных мест лидируют рестораны — около 85 мест. Чуть меньше типичная вместимость у баров/пабов (около 82 мест) и кофеен (около 80 мест). Это говорит о том, что эти форматы чаще ориентированы на посадку гостей внутри заведения и более длительное пребывание.

Средние по вместимости позиции занимают столовые (примерно 75 мест) и заведения быстрого питания (около 65 мест). При этом у быстрого питания типичная вместимость ниже, чем у ресторанов и баров, что может быть связано с более высокой оборачиваемостью мест и форматом обслуживания.

Наименьшее типичное количество посадочных мест характерно для кафе (около 60 мест), пиццерий (примерно 55 мест) и особенно булочных (около 50 мест). Для булочных это ожидаемо: часть таких заведений ориентирована на формат "взять с собой", поэтому посадочная зона может быть компактной или второстепенной.

В целом график показывает, что по вместимости категории различаются достаточно заметно: разница между лидером (ресторан) и минимальным значением (булочная) составляет примерно 35 мест. Для инвесторов это важный ориентир при выборе формата, так как вместимость напрямую влияет на требования к площади, аренде и потенциальной выручке.

Задача 5

Исследуем рейтинг заведений. Визуализируем распределение средних рейтингов по категориям заведений. Рассмотрим сильно ли различаются усреднённые рейтинги для разных типов общепита.

```
In [63]: # Сводная статистика по рейтингам всех заведений
print('Сводная статистика по столбцу rating:')
```



```
df['rating'].describe()
```

Сводная статистика по столбцу rating:

```
Out[63]: count      8402.000000
mean         4.230016
std          0.470320
min          1.000000
25%          4.100000
50%          4.300000
75%          4.400000
max          5.000000
Name: rating, dtype: float64
```

```
In [64]: # Средний рейтинг по категориям
rating_by_category = (
    df.groupby('category')['rating']
      .mean()
      .sort_values(ascending=False)
      .round(2)
)

rating_by_category
```

```
Out[64]: category
бар,паб          4.39
пиццерия         4.30
ресторан         4.29
кофейня          4.28
булочная        4.27
столовая         4.21
кафе             4.12
быстрое питание  4.05
Name: rating, dtype: float32
```

```
In [87]: # Проанализируем вариации средних рейтингов по категориям заведений
rating_stats_by_category = df.groupby('category')['rating'].describe()
rating_stats_by_category
```

```
Out[87]:
```

	count	mean	std	min	25%	50%	75%	max
category								
бар,паб	764.0	4.387697	0.380392	1.1	4.3	4.4	4.6	5.0
булочная	256.0	4.268360	0.386303	1.3	4.2	4.3	4.4	5.0
быстрое питание	603.0	4.050249	0.560949	1.1	3.9	4.2	4.3	5.0
кафе	2376.0	4.124284	0.566001	1.0	4.0	4.2	4.4	5.0
кофейня	1413.0	4.277282	0.372250	1.4	4.1	4.3	4.4	5.0
пиццерия	633.0	4.301264	0.336162	1.0	4.2	4.3	4.4	5.0
ресторан	2042.0	4.290401	0.413143	1.0	4.2	4.3	4.5	5.0
столовая	315.0	4.211429	0.454205	1.0	4.1	4.3	4.4	5.0

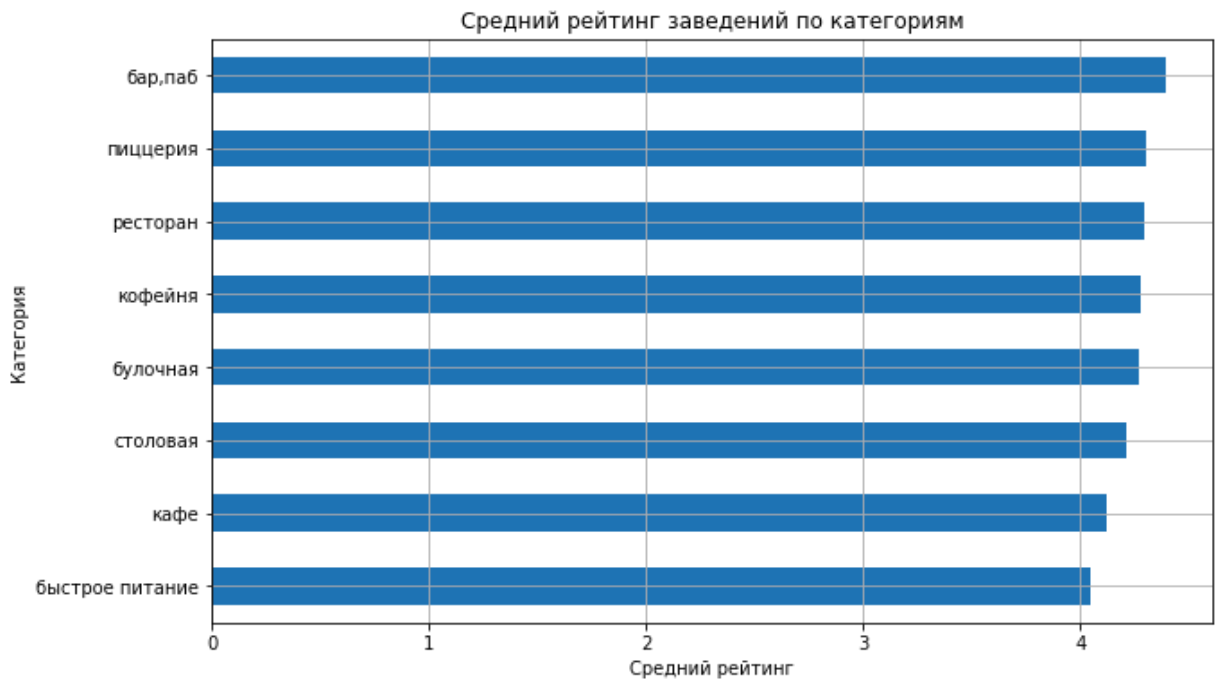
```
In [66]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(10, 6))

# Строим столбчатую диаграмму с помощью pandas через plot(kind='barh')
rating_by_category.sort_values().plot(
    kind='barh',
    title=f'Средний рейтинг заведений по категориям'
```

)

```
# Настраиваем оформление графика
plt.xlabel('Средний рейтинг')
plt.ylabel('Категория')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



Средние рейтинги заведений по категориям различаются незначительно: все значения находятся в достаточно узком диапазоне — от 4.05 до 4.39. Это говорит о том, что в целом пользователи оценивают разные форматы заведений Москвы довольно высоко и примерно на одном уровне, без резкого отрыва какой-либо категории.

Наибольший средний рейтинг у категории бар, паб (4.39), далее идут пиццерии (4.30), рестораны (4.29) и кофейни (4.28). Самый низкий средний рейтинг у категории быстрое питание (4.05), а также сравнительно низкие у кафе (4.12) и столовых (4.21).

При этом разница между максимальным и минимальным средним рейтингом составляет всего 0.34 балла, что не является очень большим разрывом по шкале от 0 до 5. Следовательно, по средним рейтингам нельзя сделать вывод о принципиально разном уровне качества между категориями — различия есть, но они умеренные.

Задача 6

Изучим, с какими данными показывают самую сильную корреляцию рейтинги заведений.

Построим и визуализируем матрицу корреляции рейтинга заведения с разными данными: его категория, положение (административный район Москвы), статус сетевого заведения, количество мест, ценовая категория и признак, является ли заведения круглосуточным.

Выберем самую сильную связь и проверим её.

In [67]:

```
# Подготовим отдельный датафрейм для phik
phik_df = df[['rating', 'category', 'district', 'chain', 'seats', 'price', 'is_24_7']]

# phik может падать на pandas nullable boolean – приводим к object
phik_df['chain'] = phik_df['chain'].astype('string').fillna('unknown').astype(object)
phik_df['is_24_7'] = phik_df['is_24_7'].astype('string').fillna('unknown').astype(object)

# Вычисляем корреляционную матрицу с использованием phi_k
correlation_matrix = phik_df.phik_matrix(verbose=False)

# Выводим результат
print('Корреляционная матрица с коэффициентом phi_k для переменной rating')
correlation_matrix.loc[correlation_matrix.index != 'rating', ['rating']].sort
```

Корреляционная матрица с коэффициентом phi_k для переменной rating

Out [67]:

	rating
price	0.262056
category	0.198739
district	0.189453
is_24_7	0.161010
chain	0.118857
seats	0.000000

In [68]:

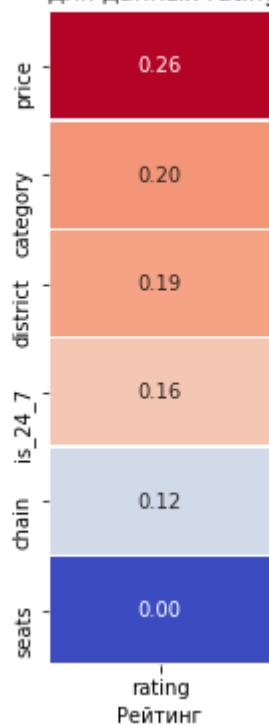
```
# Строим тепловую карту
plt.figure(figsize=(2, 6))

# Сохраняем матрицу корреляции признака rating с другими данными
data_heatmap = correlation_matrix.loc[correlation_matrix.index != 'rating']
sns.heatmap(data_heatmap,
            annot=True, # Отображаем численные значения в ячейках карты
            fmt='.2f', # Форматируем значения корреляции: два знака после точки
            cmap='coolwarm', # Устанавливаем цветовую гамму от красного (максимум) до синего (минимум)
            linewidths=0.5, # Форматируем линию между ячейками карты
            cbar=False # Отключаем цветовую шкалу
            )

# Добавляем заголовок и подпись по оси X
plt.title('Тепловая карта коэффициента phi_k \n для данных rating')
plt.xlabel('Рейтинг')

# Выводим график
plt.show()
```

Тепловая карта коэффициента ϕ_k для данных rating



Наибольшая корреляция рейтинга заведений наблюдается с ценовой категорией price (0.28). Далее по силе связи идут административный район district (0.20) и категория заведения category (0.19). Более слабая связь отмечается у признаков круглосуточного режима работы is_24_7 (0.15) и сетевого статуса chain (0.11), а количество посадочных мест seats практически не связано с рейтингом (0.00).

Наиболее заметная связь наблюдается между рейтингом и ценовой категорией заведения (price), поэтому далее проанализируем именно этот признак.

```
In [69]: # Убираем пропуски по price
price_rating_df = df[df['price'].notna()].copy()

# Табличный анализ рейтинга по ценовым категориям
price_rating_stats = (
    price_rating_df.groupby('price')['rating']
    .describe()
    .round(2)
)

price_rating_stats
```

```
Out[69]:
```

	count	mean	std	min	25%	50%	75%	max
price								
высокие	478.0	4.44	0.31	1.0	4.3	4.4	4.6	5.0
выше среднего	564.0	4.39	0.22	2.9	4.3	4.4	4.5	5.0
низкие	156.0	4.17	0.37	2.6	4.0	4.2	4.4	5.0
средние	2117.0	4.30	0.30	1.6	4.2	4.3	4.4	5.0

```
In [70]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(10, 6))
```

```
# Строим столбчатую диаграмму с помощью pandas через plot(kind='barh')
price_rating_stats['mean'].sort_values().plot(
    kind='barh',
    title=f'Средний рейтинг заведений по ценовым категориям'
)

# Настраиваем оформление графика
plt.xlabel('Средний рейтинг')
plt.ylabel('Ценовая категория')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



Средние рейтинги заведений различаются в зависимости от ценовой категории, но разница между сегментами остаётся умеренной. Самый высокий средний рейтинг у заведений категории «высокие» — 4.44, далее идут «выше среднего» — 4.39, «средние» — 4.30 и «низкие» — 4.17. Таким образом, разрыв между самой высокой и самой низкой ценовой категорией по среднему рейтингу составляет 0.27 балла.

Это означает, что более дорогие заведения в среднем получают немного более высокие оценки пользователей, однако различия нельзя назвать радикальными: во всех ценовых сегментах средний рейтинг остаётся выше 4 баллов. Следовательно, ценовая категория действительно связана с рейтингом, но эта связь скорее умеренная, а не определяющая.

Задача 7

Сгруппируем данные по названиям заведений и найдем топ-15 популярных сетей в Москве.

Для них посчитаем значения среднего рейтинга. Под популярностью понимается количество заведений этой сети в регионе.

Посмотрим к какой категории заведений они относятся. Результат сопроводим подходящими визуализациями.

In [71]:

```
# Определим топ-15 популярных сетей и средний рейтинг
chains_df = df[df['chain'] == True].copy()

# Группировка по названию сети
top_chains = (
    chains_df.groupby('name')
    .agg(
        places_count=('id', 'count'),
        avg_rating=('rating', 'mean')
    )
    .sort_values(by='places_count', ascending=False)
    .head(15)
    .round({'avg_rating': 2})
    .reset_index()
)

top_chains
```

Out[71]:

	name	places_count	avg_rating
0	шоколадница	120	4.18
1	домино'с пицца	76	4.17
2	додо пицца	74	4.29
3	one price coffee	71	4.06
4	яндекс лавка	69	3.87
5	cofix	65	4.08
6	prime	50	4.12
7	хинкальная	44	4.32
8	кофепорт	42	4.15
9	кулинарная лавка братьев караваевых	39	4.39
10	теремок	38	4.12
11	чайхана	37	3.92
12	cofest	32	3.98
13	буханка	32	4.40
14	му-му	27	4.23

In [89]:

```
tmp = df.copy() # создаем копию текущего датасета
name_tmp = 'му-му' # задаем имя столбца для проверки уникальных категорий
print(f'Заведения с одинаковым именем {name_tmp} относятся к разным категориям: {", ".join(tmp[tmp.name == name_tmp].category.unique())}')

```

Заведения с одинаковым именем му-му относятся к разным категориям: кафе, ресторан, кофейня, быстрое питание, пиццерия, столовая, бар, паб

In [73]:

```
result_top_chains = (
    chains_df.groupby('name')
    .agg(
        places_count=('id', 'count'),

```

```

        avg_rating=('rating', 'mean'),
        category=('category', lambda x: x.mode().iloc[0] if not x.mode().empty
        )
    ).sort_values(by='places_count', ascending=False)
    .head(15)
    .round({'avg_rating': 2})
    .reset_index()
)

result_top_chains

```

Out [73]:

	name	places_count	avg_rating	category
0	шоколадница	120	4.18	кофейня
1	домино'с пицца	76	4.17	пиццерия
2	додо пицца	74	4.29	пиццерия
3	one price coffee	71	4.06	кофейня
4	яндекс лавка	69	3.87	ресторан
5	cofix	65	4.08	кофейня
6	prime	50	4.12	ресторан
7	хинкальная	44	4.32	кафе
8	кофепорт	42	4.15	кофейня
9	кулинарная лавка братьев караваевых	39	4.39	кафе
10	теремок	38	4.12	ресторан
11	чайхана	37	3.92	кафе
12	cofefest	32	3.98	кофейня
13	буханка	32	4.40	булочная
14	му-му	27	4.23	кафе

In [74]:

```

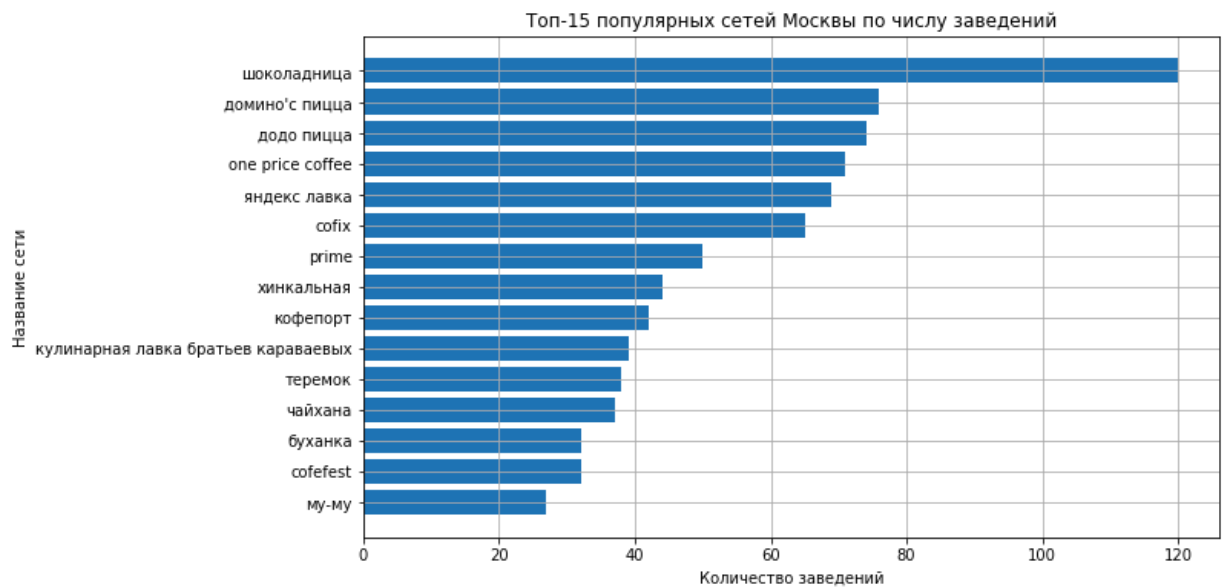
# Визуализируем Топ-15 сетей по числу заведений
# Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(10, 6))

# Строим столбчатую диаграмму с помощью pandas через plt.barh
plt.barh(top_chains.sort_values('places_count')['name'],
         top_chains.sort_values('places_count')['places_count'])

# Настраиваем оформление графика
plt.title('Топ-15 популярных сетей Москвы по числу заведений')
plt.xlabel('Количество заведений')
plt.ylabel('Название сети')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()

```



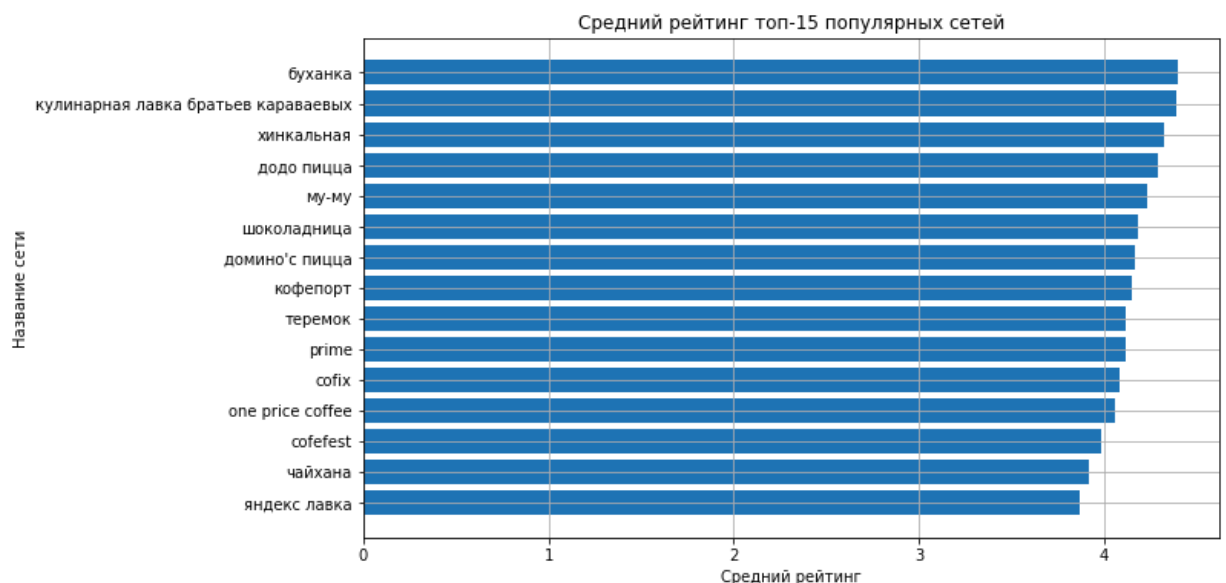
In [75]:

```
# Визуализируем средний рейтинг топ-15 сетей
# Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(10, 6))

# Строим столбчатую диаграмму с помощью pandas через plt.barh
rating_plot = top_chains.sort_values('avg_rating')
plt.barh(rating_plot['name'], rating_plot['avg_rating'])

# Настраиваем оформление графика
plt.title('Средний рейтинг топ-15 популярных сетей')
plt.xlabel('Средний рейтинг')
plt.ylabel('Название сети')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



По числу заведений в Москве среди сетевых точек лидирует «Шоколадница» (120 заведений, средний рейтинг 4.18). Далее идут крупные сети пиццерий — «Домино'с Пицца» (76, рейтинг 4.17) и «Додо Пицца» (74, рейтинг 4.29), а также кофейни One Price Coffee (71, 4.06) и Cofix (65, 4.08). Это показывает, что наиболее масштабно в городе представлены сетевые форматы кофеев и пиццерий.

По средней оценке среди топ-15 популярных сетей заметны различия, но в целом рейтинги остаются достаточно высокими — примерно в диапазоне от 3.87 до 4.40. Наиболее высокий средний рейтинг в показанном топе у «Буханки» (4.40), «Кулинарной лавки братьев Караваевых» (4.39) и «Хинкальной» (4.32), а самый низкий — у «Яндекс Лавки» (3.87). Это означает, что большая распространённость сети не всегда сопровождается более высоким пользовательским рейтингом.

Задача 8

Изучим вариацию среднего чека заведения (столбец `middle_avg_bill`) в зависимости от района Москвы.

Проанализируем цены в Центральном административном округе и других.

Посмотрим как удалённость от центра влияет на цены в заведениях. Результат сопроводим подходящими визуализациями.

```
In [76]: df['middle_avg_bill'].describe().round(2)
```

```
Out[76]: count      3149.00
mean         958.05
std          1009.73
min           0.00
25%          375.00
50%          750.00
75%         1250.00
max         35000.00
Name: middle_avg_bill, dtype: float64
```

```
In [77]: # Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(10, 6))

# Строим гистограмму с помощью pandas через plot(kind='hist')
df['middle_avg_bill'].plot(
    kind='hist',
    bins=50,
    alpha=0.75,
    edgecolor='black',
    rot=0,
)

# Настраиваем оформление графика
plt.title('Распределение значений среднего чека')
plt.xlabel('Средний чек')
plt.ylabel('Частота')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



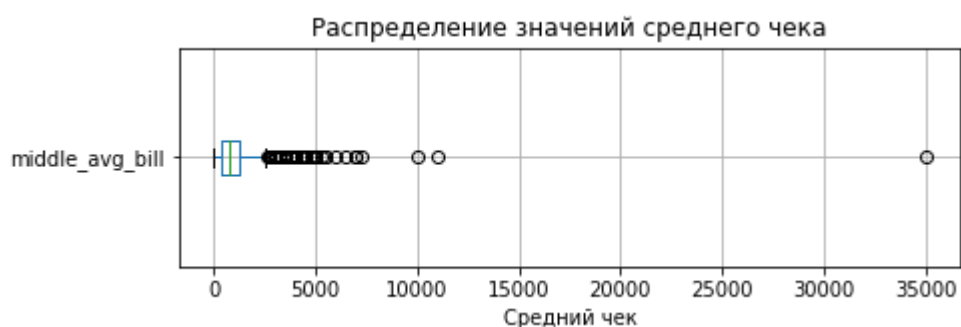
In [78]:

```
# Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(7, 2))

# Строим диаграмму размаха значений в столбце balance
df.boxplot(column='middle_avg_bill', vert=False)

# Добавляем заголовок и метки оси
plt.title('Распределение значений среднего чека')
plt.xlabel('Средний чек')

# Выводим график
plt.show()
```



По столбцу `middle_avg_bill` видно, что распределение среднего чека не является симметричным и имеет выраженную правостороннюю асимметрию. Среднее значение составляет 958 ₽, тогда как медиана — 750 ₽, то есть среднее заметно выше медианы. Это означает, что основная масса заведений сосредоточена в сегменте с более низким и средним чеком, а более дорогие заведения смещают среднее значение вверх.

Половина заведений имеет средний чек в диапазоне от 375 ₽ до 1250 ₽, что видно по квартилям (25% и 75%). При этом минимальное значение равно 0 ₽, а максимальное достигает 35000 ₽, что указывает на наличие экстремально высоких значений. Диаграмма размаха подтверждает присутствие большого количества выбросов: кроме

основной массы заведений, в данных есть отдельные объекты с очень высоким средним чеком, которые существенно отличаются от типичных значений.

Стандартное отклонение составляет 1009.73, что даже выше среднего значения, и это дополнительно говорит о сильном разбросе данных. Следовательно, для описания "типичного" уровня цен по среднему чеку корректнее опираться на медиану (750 ₽), а не только на среднее, поскольку среднее сильнее искажается дорогими заведениями-выбросами.

In [79]:

```
# Статистика по округам
bill_by_district = (
    df.groupby('district')['middle_avg_bill']
    .agg(['count', 'mean', 'median'])
    .sort_values('median', ascending=False)
    .round(2)
)

bill_by_district
```

Out[79]:

	count	mean	median
district			
Западный административный округ	306	1053.229980	1000.0
Центральный административный округ	1060	1191.060059	1000.0
Северо-Западный административный округ	157	822.219971	700.0
Северный административный округ	322	927.960022	650.0
Юго-Западный административный округ	235	792.559998	600.0
Восточный административный округ	260	820.630005	575.0
Северо-Восточный административный округ	301	716.609985	500.0
Южный административный округ	314	834.400024	500.0
Юго-Восточный административный округ	194	654.099976	450.0

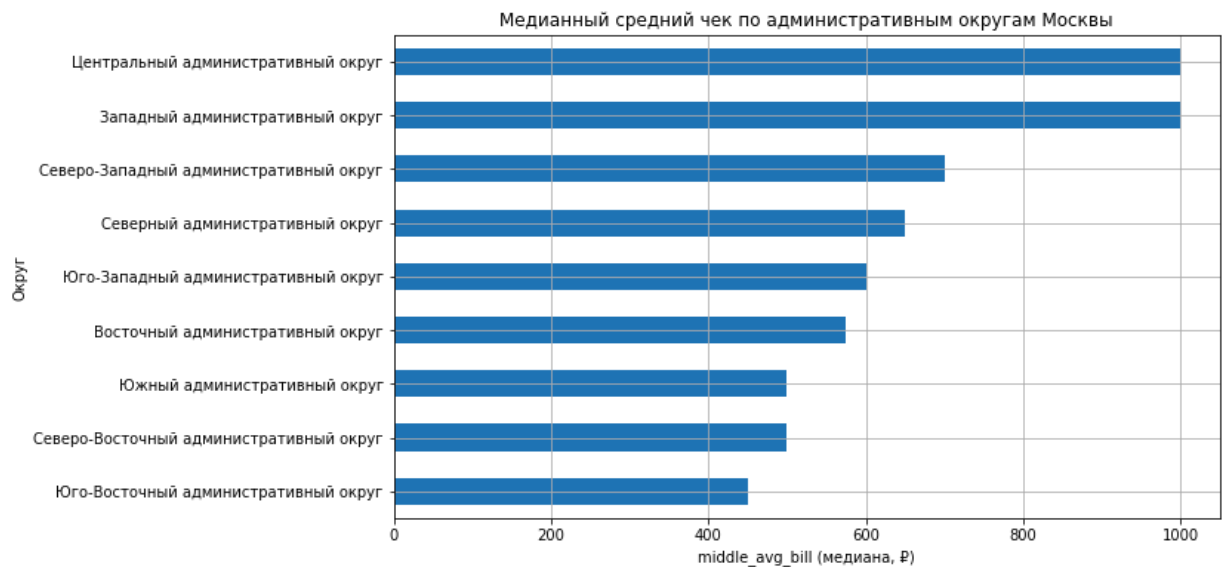
In [80]:

```
# Визуализация: медианный чек по округам
# Создаём контейнер графика matplotlib и задаём его размер
plt.figure(figsize=(10, 6))

# Строим столбчатую диаграмму с помощью pandas через plot(kind='barh')
bill_by_district['median'].sort_values().plot(kind='barh')

# Настраиваем оформление графика
plt.title('Медианный средний чек по административным округам Москвы')
plt.xlabel('middle_avg_bill (медиана, ₽)')
plt.ylabel('Округ')
# Добавляем сетку графика
plt.grid()

# Выводим график
plt.show()
```



```
In [81]: # Сравним ЦАО и остальные округа
bill_df = df[['district', 'middle_avg_bill']].copy()

bill_df['center_group'] = bill_df['district'].apply(
    lambda x: 'ЦАО' if x == 'Центральный административный округ' else 'Другие'
)

center_compare = (
    bill_df.groupby('center_group')['middle_avg_bill']
    .agg(['count', 'mean', 'median'])
    .round(2)
)

print('Сравнение ЦАО и остальных округов:')
center_compare
```

Сравнение ЦАО и остальных округов:

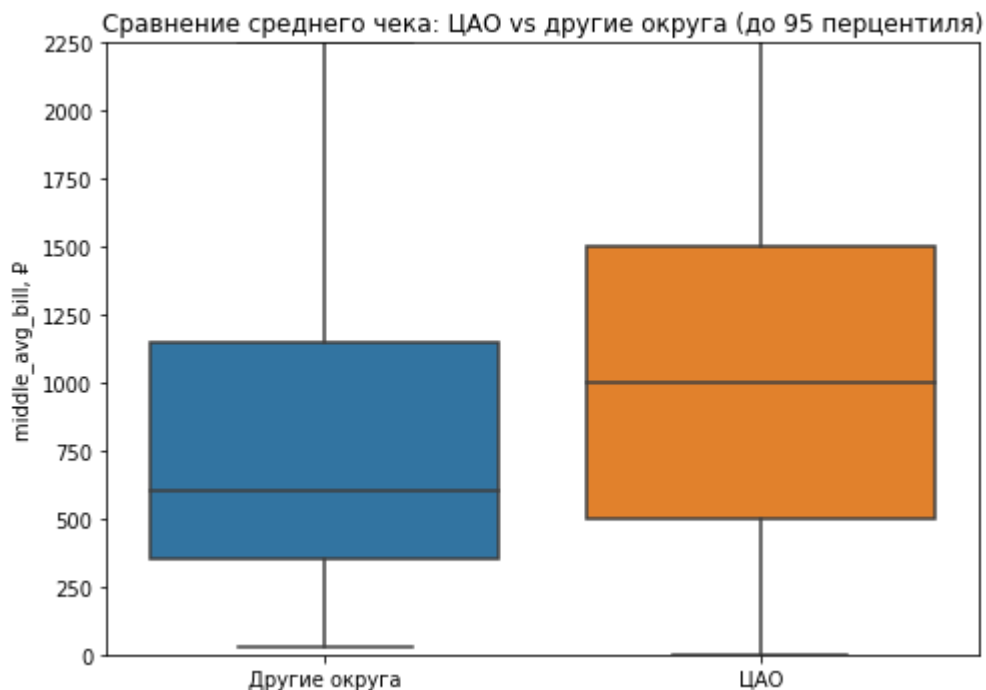
```
Out[81]:
```

	count	mean	median
center_group			
Другие округа	2089	839.820007	600.0
ЦАО	1060	1191.060059	1000.0

```
In [82]: # Создадим визуализацию
plt.figure(figsize=(7, 5))
sns.boxplot(data=bill_df, x='center_group', y='middle_avg_bill')

# Ограничиваем верхнюю границу для читаемости
plt.ylim(0, bill_df['middle_avg_bill'].quantile(0.95))

plt.title('Сравнение среднего чека: ЦАО vs другие округа (до 95 перцентиля)')
plt.xlabel('')
plt.ylabel('middle_avg_bill, ₹')
plt.tight_layout()
plt.show()
```



Анализ среднего чека по административным округам показывает, что наиболее высокий типичный уровень цен наблюдается в ЦАО и ЗАО: в обоих округах медианный чек составляет 1000 ₽. При этом по среднему значению лидирует ЦАО — около 1191 ₽, а в ЗАО средний чек немного ниже — около 1053 ₽. Это можно объяснить высокой деловой активностью, более дорогой арендой, большим туристическим и пешеходным трафиком, а также высокой концентрацией заведений более высокого ценового сегмента в центре и престижных западных районах Москвы.

Самые низкие значения среднего чека характерны для Юго-Восточного административного округа: медиана здесь составляет 450 ₽, а среднее — около 654 ₽. Также сравнительно низкий типичный чек наблюдается в Северо-Восточном и Южном округах, где медиана равна 500 ₽. Это можно интерпретировать как ориентацию заведений этих округов на более массовый и доступный ценовой сегмент, рассчитанный на повседневный спрос местных жителей, а не на премиальную аудиторию.

Разница между средним и медианой показывает наличие выбросов по среднему чеку почти во всех округах. Наиболее заметный разрыв наблюдается в Северном административном округе: среднее значение составляет около 928 ₽, а медиана — 650 ₽, то есть разница достигает почти 278 ₽. Существенный разрыв также виден в Южном округе (834 ₽ против 500 ₽) и Восточном округе (821 ₽ против 575 ₽). Это говорит о том, что в этих округах есть отдельные дорогие заведения, которые заметно повышают средний чек, хотя основная масса объектов остаётся в более доступном диапазоне.

Наиболее однородным по уровню цен выглядит Западный административный округ, где среднее (1053 ₽) и медиана (1000 ₽) близки друг к другу. Это означает, что здесь влияние выбросов минимально, а ценовой уровень заведений в целом более ровный. В ЦАО разрыв между средним и медианой тоже сравнительно небольшой (1191 ₽ против 1000 ₽), что подтверждает не только высокий, но и достаточно устойчивый уровень цен.

В целом анализ подтверждает выраженную центральную тенденцию: по мере приближения к центру и более престижным районам города средний чек растёт, тогда как в более удалённых округах преобладают заведения со средним и низким ценовым сегментом. Для инвестора это означает, что ЦАО и ЗАО подходят для форматов с более высоким чеком, тогда как ЮВАО, СВАО и ЮАО — для более массовых и доступных концепций.

Промежуточный вывод

Обобщим полученные результаты, выделив, самые важные.

- Структура рынка. В данных представлены разные форматы заведений, при этом наибольшую долю составляют массовые категории — кафе, рестораны и кофейни. Именно они формируют основную конкурентную среду в городе.
- Распределение по округам. Количество заведений заметно различается по административным округам Москвы. Это говорит о том, что плотность рынка и конкуренция существенно зависят от локации, а при выборе места для открытия заведения важно учитывать «перегретость» отдельных территорий.
- Сетевой vs несетевой формат. В целом по выборке несетевых заведений больше, однако доля сетевых сильно зависит от категории. Наиболее «сетевые» форматы — булочные, пиццерии и кофейни: эти категории чаще развиваются как сеть и характеризуются высокой стандартизацией.
- Посадочные места и выбросы. Показатель seats имеет широкий разброс и выраженные выбросы (встречаются очень крупные заведения), поэтому для описания типичного размера заведения корректнее использовать медиану, а не среднее. По типичной вместимости лидируют рестораны и бары/пабы, а наименьшее число мест характерно для булочных и пиццерий.
- Рейтинги по категориям. Средние рейтинги заведений по категориям различаются не сильно: в целом оценки пользователей достаточно высокие во всех форматах. Это означает, что сама категория заведения не является решающим фактором качества — важнее конкретная реализация (сервис, кухня, локация).
- Связь рейтинга с характеристиками. По коэффициенту ϕ_k наиболее заметная связь рейтинга наблюдается с ценовой категорией (price), а также с округом (district) и типом заведения (category). Однако значения связи умеренные, что говорит о том, что рейтинг формируется совокупностью факторов, а не одним признаком.
- Топ сетей Москвы. Среди топ-15 самых распространённых сетей лидируют форматы кофеен и пиццерий. При этом распространённость сети не гарантирует самый высокий рейтинг: у популярных сетей средние оценки могут быть как высокими, так и средними.

- Средний чек и влияние центра. Анализ middle_avg_bill показал выраженную разницу между центром и остальными округами. В ЦАО медианный чек равен 1000 ₽, тогда как в других округах — 600 ₽; среднее также выше (1191 ₽ против 840 ₽). Следовательно, близость к центру связана с более высоким уровнем цен, и при выборе концепции заведения важно учитывать, что центральные локации предполагают более дорогой сегмент.

Подведём итоги

В ходе исследования был проведён анализ рынка заведений общественного питания Москвы на основе данных из двух датасетов (информация о заведениях и данные о ценах). На этапе предобработки данные были объединены, проверены типы данных, обработаны пропущенные значения, нормализованы текстовые поля, проверены дубликаты и добавлен новый признак is_24_7 (круглосуточный режим работы).

Далее был выполнен исследовательский анализ данных: изучены категории заведений, распределение по административным округам, соотношение сетевых и несетевых заведений, количество посадочных мест, рейтинги, средний чек, а также взаимосвязь рейтинга с ключевыми характеристиками заведений. Отдельно были выделены и проанализированы наиболее популярные сети Москвы.

Главные выводы по исследовательским вопросам

Основные выводы: рынок Москвы формируют прежде всего кафе, рестораны и кофейни; в целом несетевых заведений больше, но в категориях булочные, пиццерии и кофейни доля сетевых выше. По вместимости наиболее крупные форматы — рестораны и бары/пабы, а рейтинги по категориям различаются незначительно. Наиболее заметная (но умеренная) связь рейтинга наблюдается с ценовой категорией, округом и типом заведения. Среди популярных сетей преобладают кофейни и пиццерии.

По среднему чеку выявлена важная закономерность: в ЦАО цены выше, чем в остальных округах (медиана 1000 ₽ против 600 ₽, среднее 1191 ₽ против 840 ₽). Это означает, что при выборе концепции и локации инвесторам стоит учитывать связку район + ценовой сегмент: для центра — более высокий чек, для других округов — более массовый формат.

Рекомендации

Инвесторам из фонда Shut Up and Take My Money рекомендуется выбирать формат и локацию заведения в связке: категория + район + ценовой сегмент. Если приоритет — масштабируемая и понятная бизнес-модель, стоит рассмотреть форматы, где выше доля сетевых заведений (прежде всего кофейни, пиццерии и булочные). Если цель — более высокий средний чек, перспективнее рассматривать локации в ЦАО, где уровень цен заметно выше, чем в остальных округах; для других округов более уместен массовый сегмент со средним чеком ниже. Перед запуском проекта также важно провести локальный анализ конкурентов в выбранном районе (плотность

заведений, ценовой уровень, рейтинги, доля сетей), чтобы выбрать менее перегретую нишу и точнее определить позиционирование.